

Simple Chat Application in Go and Rust
Comparison Report

Ruthwik Pala and Arjun Nandkishor Tiwari

University of the Cumberlands

MSCS 632 Advanced Programming Languages

Dr. Jay Thom

October 4, 2026

Simple Chat Application in Go and Rust

For this project, we built the same local chat application in Go and Rust and connected both backends to one browser interface. The goal was not simply to make two versions that looked the same. Building the same features in both languages gave us a practical way to compare how Go and Rust handle data, concurrency, errors, memory, and program structure. Both versions support Alice, Bob, and Charlie, direct messages, conversation history, participant and keyword filters, and a six-message concurrent simulation. They also expose the same HTTP/JSON routes, which allowed the frontend to work without knowing which backend was running.

Shared Design and Language-Specific Data Structures

Keeping the behavior consistent was important because it made the language comparison fair. Both backends use string user IDs, sequential message IDs, and timestamps in milliseconds. The main data structures are similar in purpose but use each language's standard types. Go stores users in a `map[string]User` and message history in a `[]Message` slice. Rust uses a `HashMap<String, User>` and a `Vec<Message>`. These choices felt natural in each language: the map or HashMap gives direct lookup by user ID, while the slice or Vec can grow as messages are added.

The API contract also stayed the same. Both versions provide `GET /api/users`, `POST /api/messages`, `GET /api/messages` for history and filtering, `GET /api/conversation` for a two-user conversation, and `POST /api/simulation`. Go uses JSON tags on exported struct fields, while Rust uses Serde to serialize and deserialize the corresponding data. This was useful because the frontend could send the same JSON regardless of which language was serving the request.

Implementation Comparison

Area	Go	Rust
Concurrency	Goroutines	Tokio tasks
Shared state	Shared ChatService + sync.RWMutex	Worker owns state + channels
Memory	Garbage collected	Ownership and borrowing
Errors	(value, error) and sentinel errors	Result<T, E> and typed errors
Collections	map and slice	HashMap and Vec

Concurrency and State Management

Concurrency produced the clearest difference between the two implementations. In Go, the simulation starts three goroutines, one for each simulated user. Each goroutine sends two messages in sequence. The returned message or error is placed into a buffered result channel with room for all six results. The channel is used to collect the work completed by the goroutines; it is not what protects the message history.

Figure 1

Go simulation channel and goroutine launch

```
go-backend/internal/chat/simulation.go
39 results := make(chan simulationResult, 6)
40
41 for _, user := range users {
42     user := user
43
44     go func() {
45         for _, content := range user.messages {
46             message, err := s.SendMessage(
```

The Go ChatService is shared, so concurrent sends can reach the same message slice and message-ID counter. We therefore use sync.RWMutex. A write lock protects ID assignment and appending a message, while a read lock is used when reading history. This made the responsibilities easy to separate in our design: goroutines perform concurrent work, the channel

carries simulation results, and the RWMutex protects shared mutable state (The Go Authors, n.d.-b).

Rust solves the same problem differently in our project. Instead of allowing the Tokio tasks to mutate the ChatService directly, one worker owns the mutable chat state. Cloned ChatHandle values send commands to that worker through a bounded Tokio mpsc channel. Each command also carries a one-shot reply channel so the caller can receive its result. The worker processes those commands and performs the actual mutation. Rust does provide mutexes, but our Rust implementation deliberately used an ownership-and-message-passing design rather than sharing the ChatService behind a lock (Tokio Contributors, n.d.).

Figure 2

Rust simulation tasks and cloned handles

```
rust-backend/src/simulation.rs
43 let mut tasks = JoinSet::new();
44 for (sender_id, recipient_id, name) in [
45     ("alice", "bob", "Alice"),
46     ("bob", "charlie", "Bob"),
47     ("charlie", "alice", "Charlie"),
48 ] {
49     let client = handle.clone();
50     tasks.spawn(async move {
```

This difference influenced how we reasoned about concurrency. In Go, we had to be careful that every access to shared mutable state used the correct lock. In Rust, the worker created a clear ownership boundary because the tasks did not mutate the history themselves. The Rust version required more supporting code, such as command variants and reply channels, but it also made the mutation path very explicit.

Memory Management and Performance Tradeoffs

Memory management was another major contrast. Go uses garbage collection, so unreachable heap allocations are reclaimed by the runtime. This reduces the amount of memory-management work required from the programmer, although garbage collection has its own CPU and memory costs (The Go Authors, n.d.-a). In our Go implementation, `GetMessages` takes a read lock and returns a copied slice of the stored messages. The copy lets callers work with a separate slice after the lock is released.

Rust does not use a tracing garbage collector. Instead, ownership and borrowing determine when values can be accessed and when their resources are released. In our Rust code, read-only operations can borrow the history, while a send requires mutable access. Search can inspect borrowed history and clone matching messages for the response. This did not mean Rust avoided every allocation or copy; rather, the ownership rules made those choices more explicit in the code (The Rust Project Developers, n.d.).

Figure 3

Go returns a copied history under a read lock

```
go-backend/internal/chat/chat.go
116 func (s *ChatService) GetMessages() []Message {
117     s.mu.RLock()
118     defer s.mu.RUnlock()
119
120     messages := make([]Message, len(s.messages))
121     copy(messages, s.messages)
122
123     return messages
```

Figure 4

Rust distinguishes borrowed reads from mutable access

```
rust-backend/src/chat.rs
102 /// A read-only borrow lets search inspect history without copying it first.
103 pub fn history(&self) -> &[Message] {
104     &self.history
105 }
106
107 /// Validate fully before changing state or consuming an ID.
108 pub fn send(&mut self, request: SendMessageRequest) -> Result<Message, ChatError> {
```

Go copies the full history before filtering so that filtering can continue after the read lock is released. Rust can filter borrowed history and clone the matches, but the single worker handles chat commands sequentially. Both implementations also scan the in-memory history as it grows. These are real tradeoffs in our code, but a performance ranking would require measurement rather than assumptions.

Error Handling, Syntax, and Programming Style

The error-handling styles were noticeably different. Go's `SendMessage` returns a `Message` and an error. Callers check `err` before using the result, and sentinel errors such as `ErrSameUser` identify expected validation failures. Rust represents the same idea with `Result<Message, ChatError>`. The `?` operator can propagate failures, and `match` can handle individual error variants. Both implementations reject unknown users, messages sent to the same user, and blank content before changing history.

The languages also encouraged different coding styles. Go uses packages, exported names beginning with capital letters, range loops, goroutines, channels, and `defer`. We found the syntax relatively small and direct. Rust uses crates and modules, `pub` visibility, enums, pattern matching, iterator operations, ownership, borrowing, and `async/await`. Rust's type system allowed us to express command variants and typed errors clearly, but it also meant there was more to understand while writing the program.

Neither implementation is purely object-oriented. Both use imperative control flow and group data with related methods, but neither relies on an inheritance hierarchy. Rust also uses functional-style iterator operations in parts of the search logic. This project made the paradigm discussion more concrete for us because the same feature could be expressed with a direct Go loop or with Rust's iterator and type-system features.

Development Experience and Challenges

From our experience on this project, Go had the lower barrier to entry. Starting a goroutine only requires the `go` keyword, and the standard library gave us the HTTP, JSON, synchronization, and testing tools we needed. The difficult part was making sure shared state stayed safe under concurrency. The code may look simple, but the programmer still has to identify which data is shared and protect it correctly.

Rust had a steeper learning curve. Ownership, borrowing, `Result`, `async` code, and Tokio channels made the implementation more explicit, but they also required more supporting code and more decisions about who owns mutable state. The compiler's restrictions forced us to think about those boundaries while implementing the program rather than treating memory and access as an afterthought. For this application, the worker pattern gave us one clear owner of chat state and allowed asynchronous tasks to communicate with it.

One practical challenge was keeping both applications behaviorally aligned. Standardizing on string user IDs fixed an early mismatch with the shared frontend. We also had to make sure that ID assignment and storage stayed coordinated under concurrent sends. Search behavior is almost identical, although there is still a small boundary: the Rust implementation lowercases ASCII characters while Go uses `strings.ToLower`, so non-ASCII case behavior can differ. The core English-language workflow used by the application remains the same.

Testing, Documentation, and Final Evaluation

We tested the implementations beyond the normal user workflow. The final Rust version includes 22 tests and checks 100 concurrent sends for complete storage and sequential unique IDs. The Go tests include a 60-goroutine concurrent-send test and were also run with Go's race detector. Both sides test message sending, validation, conversation selection, filters, and simulation. These tests gave us confidence in the behaviors we actually exercised, while not implying that testing can prove every possible concurrent execution correct.

The repository README documents the project structure, API routes, prerequisites, build and run commands, and test commands. The same frontend can be demonstrated with either backend because both serve the same contract on 127.0.0.1:8080. The project repository is <https://github.com/tiwariarjun91/MSCS-632-Group-Project-Simple-Chat-Application>.

Overall, implementing the same application twice made the language differences much easier to see than reading about them only in theory. Go favored a direct shared-service design with lightweight goroutines and explicit locking. Rust encouraged us to make ownership boundaries more visible and, in our implementation, to route mutations through a worker using Tokio channels. Both approaches produced the same core application, but the path to that behavior was different. The project showed us that language features affect not only syntax, but also how developers structure state, think about concurrency, handle errors, and reason about memory.

References

The Go Authors. (n.d.-a). A guide to the Go garbage collector. <https://go.dev/doc/gc-guide>

The Go Authors. (n.d.-b). Effective Go. https://go.dev/doc/effective_go

The Rust Project Developers. (n.d.). What is ownership? The Rust Programming Language.
<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

Tokio Contributors. (n.d.). Channels. Tokio tutorial. <https://tokio.rs/tokio/tutorial/channels>